WOWSCANNER

COMPREHENSIVE TECHNICAL REFERENCE MANUAL

Security Protocols · Live HUD Architecture · Status Bar Reference

Tamper Detection · Cryptographic Protocols · All 54 Audit Sections

Version 2.4.0 — 17,584 Lines of Bash

17,584	7	54	200K
Lines of Code	Security Layers	Audit Sections	PBKDF2 Rounds

cocoflan

github.com/cocoflan/wowscanner

Generated May 11, 2026 · BSD 2-Clause License

WOWSCANNER

COMPREHENSIVE TECHNICAL REFERENCE MANUAL

Security Protocols · Live HUD Architecture · Status Bar Reference

Tamper Detection · Cryptographic Protocols · All 54 Audit Sections

Version 2.4.0 — 17,584 Lines of Bash

17,584	7	54	200K
Lines of Code	Security Layers	Audit Sections	PBKDF2 Rounds

cocoflan

github.com/cocoflan/wowscanner

Generated May 11, 2026 · BSD 2-Clause License

Table of Contents

1. Introduction & Architecture

Wowscanner is a 17,584-line self-contained Bash security scanner for Debian and Ubuntu Linux. One command runs a complete audit of any Linux system:

```
sudo wowscanner --no-pentest
```

The tool combines SSH auditing, kernel hardening checks, rootkit detection, CIS benchmarking, active pentest modules, web server scanning, credential exposure detection, and LAN device discovery in a single run with a **live terminal HUD**, five professional report formats, and a seven-layer cryptographic security architecture.

Component	Description
Authentication Engine	PBKDF2-HMAC-SHA256 + SHA-256 CTR cipher. Pure Python stdlib. Embedded heredoc. No external libraries.
Script Integrity	2-phase: SHA-256 hash before passphrase + HMAC after auth. Verified backup. Tamper auto-restore.
54 Audit Sections	Bash functions covering SSH, kernel, users, ports, files, services, containers, rootkits, and more.
Live Terminal HUD	Background subshell + ANSI cursor addressing. 250ms refresh. Progress, score, findings, system LEDs.
5 Report Generators	Embedded Python heredocs (~750 lines each) producing .odt, .html, .ods, .txt, .zip with HMAC manifests.
Archive Integrity	SHA-256+SHA-512 per file, HMAC-signed INTEGRITY.txt manifest, machine-derived key.

2. Installation & First Run

2.1 Installation

```
# Install via .deb package (recommended) sudo apt-get install nmap zip python3 lynis rkhunter chkrootkit arp-scan sudo apt install ./wowscanner_2.4.0.deb sudo wowscanner # triggers first-run setup wizard # Or direct script wget https://raw.githubusercontent.com/cocoflan/wowscanner/main/wowscanner.sh chmod +x wowscanner.sh && sudo bash wowscanner.sh
```

2.2 Common Start Commands

Goal	Command	Duration
Routine audit (recommended)	sudo wowscanner --no-pentest	3–6 min
Full audit with pentest	sudo wowscanner	8–15 min
Quick hardening check	sudo wowscanner --fast-only	2–4 min
Full Lynis + rkhunter	LYNIS_FULL=true RKH_FULL=true sudo wowscanner --no-pentest	10–20 min
Help	sudo wowscanner --help	Instant

2.3 File Layout

```
/etc/wowscanner/ auth.key ← passphrase key store (mode 400, root:root) script.sig ← script integrity signature (mode 400, root:root) wowscanner.sh.backup ← verified clean backup (mode 400, root:root) wowscanner.conf ← user configuration (mode 640) /var/lib/wowscanner/ score_history.db ← score per run (for sparkline) timing_history.db ← wall times for EWMA ETA integrity_alerts.log ← tamper detection forensic log auth_failures.log ← failed passphrase attempts /var/log/wowscanner/ ← systemd timer scan output
```

3. Authentication & Cryptographic Protocols

3.1 PBKDF2 Key Derivation

PBKDF2-HMAC-SHA256 (RFC 2898) derives the 256-bit master key from the passphrase. The same algorithm is used by 1Password, LastPass, Bitwarden, macOS Keychain, and iOS data protection.

```
def pbkdf2(pw, salt, rounds): return hashlib.pbkdf2_hmac( "sha256", # PRF = HMAC-SHA256 pw.encode(), # passphrase as UTF-8 bytes binascii.unhexlify(salt), # 32 random bytes rounds, # 200,000 iterations dklen=32 # 256-bit master key )
```

Salt	32 random bytes (os.urandom)	Makes every installation unique. Prevents rainbow tables.
Rounds	200,000	~5,000 GPU guesses/sec. 10-char alphanumeric = 1,600 years to crack.
Output	256-bit master key	Ephemeral — never stored on disk.

3.2 Key Wrapping (SHA-256 CTR Cipher)

Wowscanner implements AES-256-CTR using only Python's hashlib (no external libraries). Each 32-byte block of keystream = SHA-256(key || iv || counter):

```
def aes_cbc(key, iv_bytes, data): stream = b"" counter = 0 while len(stream) < len(data): block = hashlib.sha256( key + iv_bytes + counter.to_bytes(4, "big") ).digest() # 32 bytes per block stream += block counter += 1 return bytes(x ^ y for x, y in zip(data, stream[:len(data)])) # Used to: # 1. Encrypt verification token "WOWSCANNER_OK\n\x00\x00" → auth.key:cipher # 2. Wrap data_key with master_key → auth.key:dk_wrapped
```

3.3 HMAC-SHA256 & Timing-Safe Comparison

All HMAC computations use Python's hmac module. All secret comparisons use **hmac.compare_digest()** — a C implementation that always examines all bytes regardless of where the first difference occurs, preventing timing oracle attacks.

```
# HMAC for auth.key integrity seal: mac = hmac.new(master_key_bytes, fields_bytes, hashlib.sha256).digest() # Timing-safe comparison (prevents oracle attacks): sys.exit(0 if hmac.compare_digest(computed, stored) else 1)
```

3.4 The auth.key File — Complete Format

```
# /etc/wowscanner/auth.key (mode 400, root:root) ver=2 rounds=200000 salt=<64 hex> ← 32 random bytes (unique per installation) iv=<32 hex> ← 16 random bytes (IV for verification token) hmac=<64 hex> ← HMAC-SHA256(master_key, salt||iv||cipher||dk_iv||dk_wrapped) cipher=<32 hex> ← SHA-256-CTR-encrypt(master_key, iv, "WOWSCANNER_OK\n\x00\x00") dk_iv=<32 hex> ← 16 random bytes (IV for data key wrapping) dk_wrapped=<64 hex>← SHA-256-CTR-encrypt(master_key, dk_iv, data_key) key_id=<16 hex> ← SHA-256(data_key)[:8 bytes] fingerprint rk_hash=<64 hex> ← SHA-256(recovery_key_value) — for recovery verification
```

3.5 Authentication Protocol — 5 Phases

Phase	Name	Implementation
Phase 1	Passphrase input	read -rsp disables terminal echo. 3-attempt lockout. Min 10 chars.
Phase 2	Master key derivation	PBKDF2(passphrase, salt, 200000) → 256-bit master key (~2-5 sec)
Phase 3	HMAC verification	HMAC(master_key, salt||iv||cipher||dk_iv||dk_wrapped) vs stored. Timing-safe.

Phase	Name	Implementation
Phase 4	Plaintext check	Decrypt cipher → must equal "WOWSCANNER_OK". Defence-in-depth.
Phase 5	Data key unwrap	AES-CTR-decrypt(master_key, dk_iv, dk_wrapped) → WS_DATA_KEY exported

4. Script Integrity System

Wowscanner verifies its own script file on every startup. This defends against **tool poisoning** — an attacker who modifies the security scanner itself to hide findings or add a backdoor. Since Wowscanner runs as root, a compromised scanner is more dangerous than a compromised application.

4.1 Phase 1 — SHA-256 Verification (before passphrase prompt)

```
# Runs BEFORE passphrase prompt — catches naive modifications immediately _current_hash=$(sha256sum "$0" | awk '{print $1}') _stored_hash=$(grep "^script_hash=" script.sig | cut -d= -f2-) if [[ "$_current_hash" != "$_stored_hash" ]]; then _ws_script_tamper_halt "Hash mismatch — script modified since signing" fi
```

4.2 Phase 2 — HMAC Signature (after authentication)

```
# After passphrase succeeds and WS_DATA_KEY is available: _expected_sig=$(_ws_crypto hmac "$WS_DATA_KEY" "$_stored_hash") # Cannot be forged without the data key (which requires the passphrase) if ! _ws_crypto compare "$_expected_sig" "$_stored_sig"; then _ws_script_tamper_halt "HMAC mismatch — script.sig may have been forged" fi
```

4.3 The script.sig File & Verified Backup

```
# /etc/wowscanner/script.sig (mode 400, root:root) script_hash=<64 hex> ← SHA-256 of the script file sig_hmac=<64 hex> ← HMAC-SHA256(data_key, script_hash) — unforgeable without key signed_by=<16 hex> ← WS_KEY_ID fingerprint signed_path=/usr/bin/wowscanner backup_path=/etc/wowscanner/wowscanner.sh.backup backup_hash=<64 hex> ← SHA-256 of backup — verifies backup before restore
```

4.4 Tamper Detection — All Three Scenarios

Scenario	Detection Condition	Response	Severity
Scenario A (most common)	Script modified. SHA-256(backup) matches backup_hash in sig.	Green box: "✔ CLEAN BACKUP AVAILABLE" Interactive: "Restore from backup? [Y/n]" Auto-restores if accepted. Logs backup=CLEAN.	LOW — run reset-auth after restore to re-sign
Scenario B (serious)	Script modified AND backup also modified. SHA-256(backup) ≠ backup_hash.	Red box: "■ BACKUP ALSO TAMPERED — HIGH SEVERITY INCIDENT" No restore offered. Download fresh from GitHub required. Logs backup=ALSO_TAMPERED.	HIGH — forensic investigation required
Scenario C (no backup)	Script modified. No backup file found.	Yellow box: "■ No verified backup available" Manual recovery instructions shown. Logs backup=NONE.	MEDIUM — manual download and re-sign

5. THE LIVE HUD — Complete Reference

The Wowscanner live HUD (Heads-Up Display) is a real-time status panel that remains pinned to the top of the terminal throughout the entire scan. It refreshes every 250 milliseconds without disturbing the scan output that scrolls below it.

5.1 What the HUD Is and How It Works

The HUD consists of a fixed band of terminal rows at the top of the screen. Scan output scrolls in a protected region below. The HUD itself never scrolls away.

- A **background subshell** is forked at scan start. It reads two IPC state files every 250ms and redraws the HUD.
- ANSI escape codes position the cursor, wipe each row, and write new content — all in one atomic write to /dev/tty.
- A **scroll region** is set so only the rows below the HUD participate in scrolling — rows 1 to N (the HUD rows) are physically protected by the terminal emulator.
- When the scan ends, the scroll region is reset to the full screen and the cursor is restored.

5.2 Full HUD Layout — Annotated Diagram

The complete HUD as it appears during a scan on a typical 80-column terminal (shown here at representative width). The scroll region for scan output begins immediately below Row N (the bottom separator):

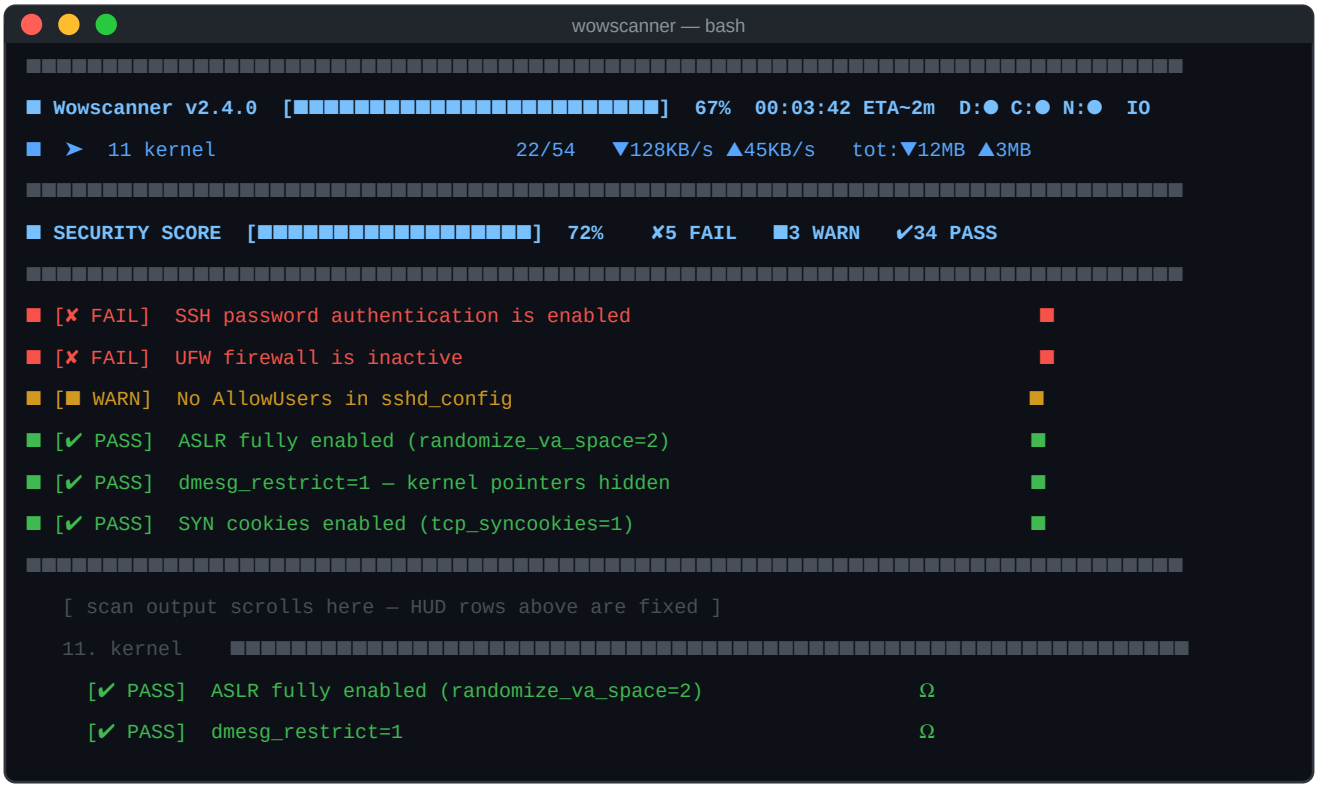

Figure 1 — The Wowscanner live HUD. Rows 1–N are fixed; scan output scrolls below.

Row	Name	Content
Row 1	Top separator	Thin horizontal line spanning the full terminal width. Colour matches progress bar.
Row 2	Progress & system	Progress bar, percentage, elapsed time, ETA, disk/CPU/net LEDs, I/O rates.
Row 3	Active section	Current section name, section counter (N/54), per-tick net rates, cumulative totals.

Row	Name	Content
Row 4	Middle separator	Same as Row 1.
Row 5	Security score	Live score bar, percentage, FAIL/WARN/PASS counters — updates after every check.
Row 6	Lower separator	Same as Row 1.
Rows 7–N-1	Findings checklist	Rolling list of recent PASS/FAIL/WARN findings. Newest at bottom.
Row N	Bottom separator	Marks the end of the HUD. Scroll region begins at Row N+1.

5.3 Row 1 & 4 — Separator Lines

The separator rows are purely visual. They mark the top and middle boundaries of the HUD. Their colour matches the current progress bar colour — cycling from cyan (0–24%) → bright yellow (25–49%) → yellow (50–74%) → green (75–99%) → bold green (100%).

```
# Separator construction: printf -v _sep '%*s' "$(( _cols - 2 ))" '' _sep="${_sep// /■}" # fill with box-drawing characters printf '\033[1;1H\033[2K%s■%s■%s' "$_bc" "$_sep" "$_R" >/dev/tty # \033[1;1H = go to row 1, col 1 # \033[2K = erase entire row (prevents old content showing through) # $bc = colour escape for current progress level
```

5.4 Row 2 — Progress Bar & System Monitors

Row 2 is the most information-dense row. It contains seven distinct data elements arranged to fit in the available terminal width:

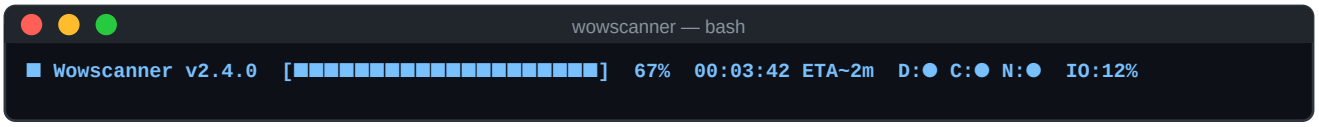

Figure 2 — Row 2 zoomed in. Seven data elements from left to right.

Progress Bar

The progress bar shows how far through the scan Wowscanner is. Width is adaptive:

```
# Bar width: 20% of terminal columns, clamped to 16..32 characters _bw=$(( _cols * 20 / 100 )) [[ "$_bw" -lt 16 ]] && _bw=16 [[ "$_bw" -gt 32 ]] && _bw=32 # Fill calculation: _filled=$(( _pct * _bw / 100 )) _bar="" # build the bar string: for _i in {0.._filled-1}: _bar+="■" # filled blocks for _i in {_filled.._bw-1}: _bar+="■" # empty blocks # Colour progression by percentage: # 0–24%: Cyan (scan just started) # 25–49%: Bright yellow # 50–74%: Yellow # 75–99%: Green # 100%: Bold green (scan complete)
```

Progress Percentage

The percentage shown is not just a simple "sections completed / total" ratio. Each section has a **weight** proportional to its expected runtime. Progress = cumulative weight of completed sections / total active weight:

```
# Weight examples (out of 925 total): # 0b (nikto web scan): 120 ← heaviest, takes longest # 0a (nmap full scan): 90 # 14b (rkhunter): 70 # 15 (lynis): 50 # 1 (sysinfo): 3 ← lightest, runs fast # Progress % = _PROGRESS_CUM[current_section] * 100 / 925 # (or / 540 when --no-pentest removes sections 0a-0e) # Within a section: animated by linear interpolation between # floor% (section started) and ceil% (section would complete)
```

Elapsed Time & ETA

The elapsed time is shown in HH:MM:SS format. The ETA is calculated using EWMA (Exponentially Weighted Moving Average) on previous scan durations.

```
# Elapsed: printf -v _efmt "%02d:%02d:%02d" \ "$(( _elapsed/3600 ))" "$(( (_elapsed%3600)/60 ))" "$(( _elapsed%60 ))" # ETA (shows only if past runs exist in timing_history.db): if [[ -n "$_eta_raw" ]]; then _eta_s="${_eta_raw%%|*}" # predicted total seconds _rem=$(( _eta_s - _elapsed )) # remaining seconds _rm=$(( _rem/60 )) _rs=$(( _rem%60 )) _eta=" ETA~${_rm}m${_rs}s" # e.g. " ETA~2m34s" fi
```

System Activity LEDs: D● C● N●

Three LED indicator dots show real-time system activity. The colour of each dot changes continuously based on actual measured activity:

LED	Metric	Source	Colour Range
D●	Disk I/O	Reads /proc/diskstats	Blue (idle) → Cyan → Green → Yellow → Red (heavy)

LED	Metric	Source	Colour Range
C●	CPU utilisation	Reads /proc/stat	Blue (idle) → Cyan → Green → Yellow → Red (100%)
N●	Network activity	Reads /proc/net/dev	Blue (idle) → Cyan → Green → Yellow → Red (busy)

```
# LED colour calculation (same formula for disk, CPU, net): if [[ "$pct" -ge 90 ]]; then col="\033[1;31m"
# bright red elif [[ "$pct" -ge 70 ]]; then col="\033[0;33m" # yellow elif [[ "$pct" -ge 40 ]]; then
col="\033[0;32m" # green elif [[ "$pct" -ge 10 ]]; then col="\033[0;36m" # cyan else col="\033[2;34m" #
dim blue (idle) fi # Disk I/O — reads /proc/diskstats (no fork, pure bash): while IFS= read -r _dsl; do
read -r _ _ _df3 _df4 _ _ _ _df8 _ <<< "$_dsl" [[ "$_df3" == "loop"* || "$_df3" == "ram"* ]] && continue
_ds_now=$(( _ds_now + _df4 + _df8 )) done < /proc/diskstats _ds_delta=$(( _ds_now - _ds_prev ))
_led_disk_pct=$(( _ds_delta * 100 / 220 )) # normalise to 0-100%
```

IO:N% CPU:N% — Percentage Readouts

Alongside the LEDs, numeric percentages give exact values. IO% is disk activity normalised to 0–100. CPU% is the fraction of CPU time not spent idle.

```
# CPU utilisation from /proc/stat: IFS= read -r _cpu_l < /proc/stat read -r _ _wu _wn _ws _wi _wio _ <<<
"$_cpu_l" _ct=$(( wu+wn+ws+wi+wio )) # total jiffies _ci=$(( wi+wio )) # idle + iowait jiffies _dt=$((
_ct - _cpu_tot_prev )) _di=$(( _ci - _cpu_idl_prev )) _led_cpu_pct=$(( (_dt - _di) * 100 / _dt )) #
utilisation %
```

▼RX ▲TX — Network Data Rates

The per-tick (250ms) network receive and transmit rates for the primary network interface. Auto-scaled from B/s to KB/s to MB/s:

```
# Read from /proc/net/dev (no fork): while IFS=: read -r _nif _nr; do [[ "$_nif" != "${_PROG_NET_IFACE}"
]] && continue read -r _nrxb _ _ _ _ _ _ _ _ntxb _ <<< "$_nr" done < /proc/net/dev # Rate = (current -
previous) per 250ms tick × 4 to get per-second _rx_delta=$(( _nrxb - _prev_rx )) _led_net_rx=$((
_rx_delta * 4 )) # Display scaling: if [[ "$_rx" -ge 1048576 ]]; then printf -v _rxs "%3dMB/s"
"$((_rx/1048576))" elif [[ "$_rx" -ge 1024 ]]; then printf -v _rxs "%3dKB/s" "$((_rx/1024))" else printf
-v _rxs "%3d B/s" "$_rx"; fi
```

5.5 Row 3 — Active Section & Network Rates

Row 3 shows which audit section is currently executing along with the section counter, per-tick data rates, and cumulative network totals since scan start:

```
Row 3 layout: ■ ➤ <section_label_up_to_32_chars> N/54 ▼RX ▲TX tot:▼X ▲Y ■ Example: ■ ➤ 11 kernel 22/54 ▼128KB/s ▲45KB/s tot:▼12MB ▲3MB ■
```

Section Label

The section label is truncated to 32 characters maximum. It comes from the _PROGRESS_LABELS array which maps every section number to a short name:

```
_PROGRESS_LABELS=( "0a pentest-enum" "0b pentest-web" "0c pentest-ssh" "0d pentest-sqli" "0e pentest-stress" "1 sysinfo" "2 updates" "3 users" "4 password" "5 ssh" "6 firewall" "7 ports" "8 permissions" "9 services" "10 logging" "11 kernel" "12 cron" "13 packages" # ... 36 more labels ... "17z exposure" )
```

Section Counter (N/54)

Shows how many sections have been started out of the total active count. With --no-pentest this shows N/49 (five pentest sections skipped). With --fast-only it also shows N/49. The counter is updated when _progress_start() is called at the beginning of each section.

Cumulative Network Totals (tot:▼X ▲Y)

The "tot:" values show total bytes received and transmitted by the machine since the scan started (not just by Wowscanner itself — the entire system). This is useful for detecting unexpected network activity during a scan.

```
# Baseline captured at scan start: if [[ "$_net_base_set" -eq 0 && "${_nrxb:-0}" -gt 0 ]]; then _net_rx_base="${_nrxb}" # total RX bytes at scan start _net_tx_base="${_ntxb}" # total TX bytes at scan start _net_base_set=1 fi # Each tick: cumulative = current - baseline _net_rx_total=$(( nrxb_now - _net_rx_base )) _net_tx_total=$(( ntxb_now - _net_tx_base )) # Displayed as: tot:▼12MB ▲3MB # Format: B → KB → MB → GB depending on magnitude
```

5.6 Row 5 — Live Security Score

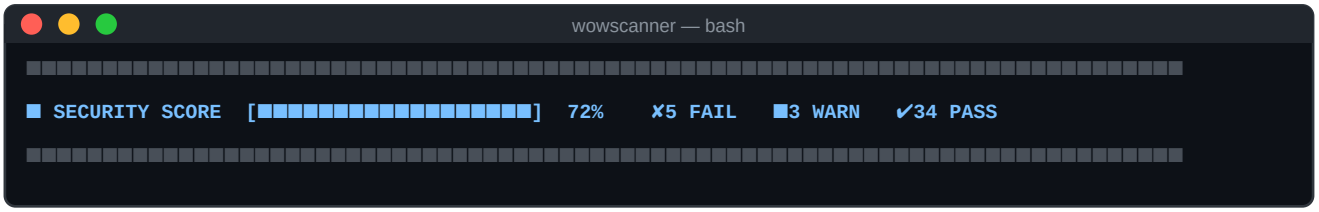

Figure 3 — Row 5 (Security Score row) with colour-coded score bar and counters.

Row 5 is the live security score line. It updates after every single check (every pass(), fail(), or warn() call), giving real-time visibility into the overall security posture:

```
Row 5 layout: ■ SECURITY SCORE [■■■■■■■■■■■■■■■■■■■■] 72% ✘5 FAIL ■3 WARN ✔34 PASS ■ # Score bar
width: fixed 18 characters _sbw=18; _sfilled=$(( _sec_pct * 18 / 100 )) # Score percentage: _sec_pct=$((
_mon_pass * 100 / _mon_total )) # Score colour: if [[ "$_sec_pct" -ge 80 ]]; then _sc="\033[1;32m" # green
elif [[ "$_sec_pct" -ge 50 ]]; then _sc="\033[1;33m" # yellow else _sc="\033[1;31m" # red fi
```

Score Calculation

Security Score %	= PASS count / (PASS + FAIL + WARN) × 100	Checks only — not per finding
GOOD (green)	≥ 80%	System is well-hardened
MODERATE (yellow)	50–79%	Improvements needed
CRITICAL (red)	< 50%	Serious security issues present

Counter Update Protocol

The counters are maintained in the monitor state IPC file (a /tmp/wowsc_mon_XXXXXX file). Every pass/fail/warn call atomically updates this file via a tmp+mv rename:

```
monitor_finding() { local _kind="$1" _text="$2" # kind = PASS|FAIL|WARN # Update in-memory counters case
"$_kind" in PASS) _mon_pass_count=$(( _mon_pass_count + 1 )) ;; FAIL) _mon_fail_count=$(( _mon_fail_count
+ 1 )) ;; WARN) _mon_warn_count=$(( _mon_warn_count + 1 )) ;; esac # Write new state atomically (tmp file
+ mv rename) local _tmp="${_PROGRESS_MONITOR_STATE}.tmp" printf "SCORE:%d:%d:%d\n" \ "$_mon_pass_count"
"$_mon_fail_count" "$_mon_warn_count" > "$_tmp" # Append existing + new entry (with ring-buffer eviction)
grep "^ENTRY:" "$_PROGRESS_MONITOR_STATE" >> "$_tmp" 2>/dev/null || true printf "ENTRY:%s:%s\n" "$_kind"
"${_text:0:100}" >> "$_tmp" mv -f "$_tmp" "$_PROGRESS_MONITOR_STATE" }
```

5.7 Rows 7–N — Rolling Findings Checklist

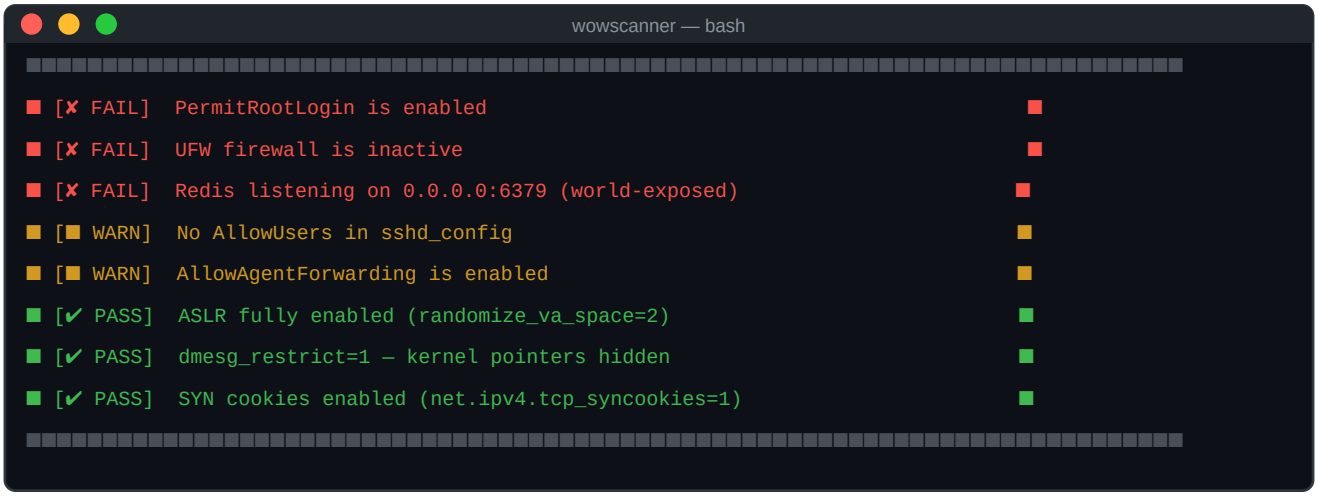

Figure 4 — The rolling findings checklist. Newest finding at bottom. FAILs are red, WARNs yellow, PASSes green.

The checklist rows display recent PASS/FAIL/WARN findings as they are generated. The checklist behaves as a scrolling ring buffer with intelligent eviction:

Capacity	40 entries maximum (configurable via _MON_CAP)
Scroll direction	Newest entry added at bottom. Old entries scroll up and off the top.
Eviction order	When full: PASSes evicted first (oldest first), then WARNs, then FAILs last.
Why this order	FAILs are the most important — they stay visible as long as possible.
Display rows	Checklist_rows = clamp(terminal_rows – 10, 4, 12). Typically 8 rows.
Colour coding	[✘ FAIL] = red · [■ WARN] = yellow · [✔ PASS] = green
Truncation	Finding text truncated to fit terminal width (cols – 13 characters).

How Many Rows?

```
# Checklist rows are calculated from terminal height: _cl=$(( _PROGRESS_ROWS - 10 )) [[ "$_cl" -lt 4 ]] && _cl=4 # minimum 4 checklist rows [[ "$_cl" -gt 12 ]] && _cl=12 # maximum 12 checklist rows _PROGRESS_CHECKLIST_ROWS=$_cl # For a 24-row terminal: 24-10=14 → clamped to 12 rows # For a 80-row terminal: 80-10=70 → clamped to 12 rows # For a 18-row terminal: 18-10=8 → 8 rows # Total HUD height: _PROGRESS_HUD_ROWS=$(( 7 + _cl )) # Row 1 (sep) + Row 2 (bar) + Row 3 (label) + Row 4 (sep) # + Row 5 (score) + Row 6 (sep) + CL_ROWS + Row N (bot sep)
```

Scroll Region — How Scan Output Is Protected

The ANSI scroll region escape ensures scan output from log() and echo never overwrites the HUD rows. The terminal emulator enforces this in hardware:

```
# Set scroll region: rows _scroll_start .. _PROGRESS_ROWS scroll # Rows 1 .. _PROGRESS_HUD_ROWS are PROTECTED — cannot be scrolled away _scroll_start=$(( _PROGRESS_HUD_ROWS + 1 )) printf '\033[%d;%dr' "$_scroll_start" "$_PROGRESS_ROWS" >/dev/tty # Move cursor to first scrolling row so output lands there: printf '\033[%d;1H' "$_scroll_start" >/dev/tty # On scan finish: reset to full-screen scroll printf '\033[r' >/dev/tty # no args = full screen printf '\033[?25h' >/dev/tty # restore cursor visibility
```

5.8 The Final Security Monitor Panel

When the scan completes and all report generators have finished, Wowscanner removes the live HUD and prints a static full-page security monitor panel. This panel shows all accumulated findings categorised by severity:

```
wowscanner — bash

■       REALTIME SECURITY MONITOR — FINAL STATE                        ■

■ SECURITY SCORE  [■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■]  85%  ✗5  ■3  ✔47      ■

■ ✗  FAILURES  (5)                                                    ■
■                                                                       ■
■ [✗ FAIL]  SSH password authentication is enabled                    ■
■ [✗ FAIL]  UFW firewall is inactive                                  ■
■ [✗ FAIL]  Redis listening on 0.0.0.0:6379                          ■
■ [✗ FAIL]  17 pending security updates                               ■
■ [✗ FAIL]  World-writable file found in /etc                         ■

■ ■  WARNINGS  (3)                                                    ■
■                                                                       ■
■ [■ WARN]  No AllowUsers configured in sshd_config                   ■
■ [■ WARN]  AllowAgentForwarding is enabled                           ■
■ [■ WARN]  auditd has only 12 rules configured                       ■

■ ✔  PASSING CHECKS  (showing last 20 of 47)                          ■
■                                                                       ■
■ [✔ PASS]  ASLR fully enabled (randomize_va_space=2)                  ■
■ [✔ PASS]  dmesg_restrict=1 — kernel pointers hidden from non-root   ■
■ [✔ PASS]  SYN cookies enabled (tcp_syncookies=1)                    ■
■ ...  44 more PASS findings not shown  ...                           ■
```

Figure 5 — The final security monitor panel printed after all report generators complete.

Panel Sections

Header	Box title with "REALTIME SECURITY MONITOR — FINAL STATE". Border colour matches score: green ≥80%, yellow 50–79
Score row	Score bar (width 30 chars), percentage, and FAIL/WARN/PASS counts.
FAILURES section	ALL FAILs shown. No cap. Red border and text.
WARNINGS section	ALL WARNs shown. Amber border and text.
PASSING section	Last 20 PASSes shown (total count displayed). Green border.

Recommended Next Steps (shown after the panel)

```
■■■ RECOMMENDED NEXT STEPS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■ ■ ① Fix all FAIL
items immediately ■ ■ ② Review WARN items — apply where relevant ■ ■ ③ sudo wowscanner baseline —
snapshot PASS findings ■ ■ ④ sudo wowscanner harden — apply sysctl hardening ■ ■ ⑤ sudo wowscanner
install-timer — weekly automated scans ■ ■ ⑥ less wowscanner_findings_<TS>.txt — browse findings ■ ■ ⑦
Aim for Lynis hardening index >= 80 ■ ■ ⑧ Set up monitoring: auditd + fail2ban + OSSEC ■ ■ ■
```

5.9 Background Subshell & IPC Protocol

The HUD is rendered by a background subshell that runs concurrently with the audit sections. Data flows from the main process to the subshell through two atomic IPC state files:

File	Purpose	Format & Protocol
/tmp/wowsc_prog_XXXXXX	Progress state	Single line: label|floor|ceil|step|total|cl_rows|hud_rows Written by _progress_write_state() via .tmp+mv (atomic rename). Read by background subshell every 250ms.
/tmp/wowsc_mon_XXXXXX	Monitor state	Line 1: SCORE:pass:fail:warn Lines 2+: ENTRY:KIND:text (one per finding) Written by monitor_finding() via .tmp+mv (atomic rename). Read by background subshell each tick.

Why Atomic Writes?

The background subshell reads the state files every 250ms. A non-atomic write (open→write→close) would allow the subshell to read a partially-written file and render garbage. The tmp+mv pattern on the same filesystem is an atomic rename — the subshell either reads the old complete file or the new complete file, never a partial write.

Refresh Loop (250ms interval)

```
_progress_bg_refresh() { local _sf="$1" # progress state file local _total="$2" # total active section
count local _t_start="$3" # scan start epoch local _eta_raw="$4" # EWMA ETA string local _mf="$5" #
monitor state file while true; do # Read current progress state (atomic) IFS= read -r _line < "$_sf"
2>/dev/null || true # Parse: label|floor|ceil|step|total|cl_rows|hud_rows IFS='|' read -r _label _floor
_ceil _step _tot _cl _hud <<< "$_line" # Calculate animated progress within current section _elapsed=$((
$(date +%s) - _t_start )) _pct=$(( _floor + (_ceil - _floor) * _elapsed / _exp_secs )) [[ "$_pct" -gt $((
_ceil - 1 )) ]] && _pct=$(( _ceil - 1 )) # Read /proc metrics (no forks) _read_disk_led # updates
_led_disk_pct, _led_disk_col _read_cpu_led # updates _led_cpu_pct, _led_cpu_col _read_net_led # updates
_led_net_rx, _led_net_tx, _led_net_col # Render HUD (all rows in one write to /dev/tty)
_progress_draw_band "$_label" "$_pct" "$_elapsed" "$_PROGRESS_COLS" "$_eta_raw" sleep 0.25 # 250ms
refresh interval done }
```

5.10 EWMA ETA Prediction Algorithm

The ETA (Estimated Time of Arrival) displayed in Row 2 is calculated using an Exponentially Weighted Moving Average (EWMA) over the last 10 scan durations. This means recent scans are weighted more heavily than older ones:

```
get_timing_eta() { local _alpha=40 # α × 100 (so α = 0.40) local _ewma=0 _first=1 _count=0 # Read last 10 runs from timing_history.db while IFS='|' read -r _ts _w _rest; do _w=$(safe_int "$_w") # wall seconds for this run [[ "$_w" -eq 0 ]] && continue if [[ "$_first" -eq 1 ]]; then _ewma="$_w" # seed with first value _first=0 else # EWMA: new = α×actual + (1-α)×old # Integer: (_alpha×_w + (100-_alpha)×_ewma) / 100 _ewma=$(( (_alpha * _w + (100 - _alpha) * _ewma) / 100 )) fi _count=$(( _count + 1 )) done < <(tail -10 "$TIMING_HISTORY_DB" 2>/dev/null) echo "${_ewma}|${_count}" # "predicted_secs|run_count" }
```

With α=0.40: each new run contributes 40% weight, the accumulated history contributes 60%. After 3–4 scans the prediction is typically within 15–20% of actual duration.

Example EWMA Calculation

```
# 3 past runs: 412s, 398s, 387s (oldest to newest) ewma_1 = 412 # seed ewma_2 = (0.40×398 + 0.60×412) = 406 # 40% new ewma_3 = (0.40×387 + 0.60×406) = 399 # 40% newer # ETA displayed in Row 2: remaining = 399 - elapsed_so_far → "ETA~6m39s" (if elapsed = 0s) → "ETA~done" (if elapsed >= predicted)
```

5.11 Terminal Compatibility & Fallbacks

Condition	Behaviour
HUD enabled when?	isatty(stdout) = true AND $TERM != "dumb". Piped/redirected output disables the HUD.
Unicode check	Tests printf "\u2588" at startup. If it fails, ASCII fallback: ■→# ■→- ●→* ▲→^ ▼→v
Minimum terminal	60 columns minimum. Below this, bar width is clamped to 16 chars and some elements may be truncated.
Colour support	Assumes ANSI colour support. $TERM=dumb disables HUD entirely (no colour output).
No /dev/tty?	If /dev/tty is not accessible, HUD is silently disabled. Scan runs normally without HUD.
Width detection	stty size read at startup and on SIGWINCH to adapt to terminal resize.

6. The 54 Audit Sections

Each section runs as a Bash function. Every individual check calls pass(), fail(), warn(), or info(). The Ω character at the end of each PASS/FAIL/WARN line is the parser anchor for all five report generators.

Pentest Warning

- Sections 0a–0e use REAL attack tools (nmap, nikto, hydra, sqlmap, stress-ng, hping3)
- Only run on systems you own or have explicit written permission to test
- Skip with: sudo wowscanner --no-pentest

#	Label	What Is Checked	Typical
0a	pentest-enum	nmap -sV -O -A -p- --open + enum4linux SMB enum	~90s
0b	pentest-web	nikto on all HTTP/HTTPS ports	~120s
0c	pentest-ssh	hydra 10 credential pairs against SSH	~30s
0d	pentest-sqli	sqlmap level=1 on HTTP ports	~60s
0e	pentest-stress	stress-ng 30s + hping3 SYN flood 5s	~40s
1	sysinfo	hostname, OS, kernel, uptime, CPU, RAM, disk, zombies	~3s
2	updates	apt pending, security updates, unattended-upgrades	~5s
3	users	UID-0 accounts, sudo group, inactive accounts, authorized_keys	~4s
4	password	/etc/shadow hashes, login.defs, PAM complexity	~2s
5	ssh	sshd -T full config dump (cached for section 17q)	~3s
6	firewall	ufw status, iptables rules, firewalld	~2s
7	ports	ss -tlnp/ulnp, bind-address risk per service	~3s
8	permissions	find SUID/SGID, world-writable /etc, debsums -c	~40s
9	services	systemctl --failed, services running as root	~2s
10	logging	rsyslog/auditd/journald status, auth.log age/perms	~3s
11	kernel	sysctl ASLR/dmesg/kptr/syncookies/ip_forward/redirects	~2s
12	cron	/etc/cron*, user crontabs, world-writable cron files	~3s
13	packages	dpkg count, debsums -c integrity, pip/npm audit	~30s
13c	hardening	fail2ban, pam_faillock, su restriction, /tmp noexec	~15s
13d	net-container	docker daemon.json, socket perms, containers root	~5s
14b	rkhunter	chkrootkit -q + rkhunter --update --check	~45s
14	mac	aa-status enforce/complain count, getenforce/sestatus	~2s
15	lynis	lynis audit system --quick (full with LYNIS_FULL=true)	~30s
16a	lan-scan	arp-scan -l or nmap -sn → JSON host map	~15s
16	portscan	nmap 3 random 500-port windows + 1–1024	~25s
17b	failed-logins	auth.log SSH brute-force patterns, fail2ban bans	~4s

#	Label	What Is Checked	Typical
17c	env-security	umask, PATH world-writable entries, LD_PRELOAD	~2s

#	Label	What Is Checked	Typical
17d	usb-audit	lsusb storage/network, usb-storage lsmod	~2s
17e	ww-deep	find world-writable in /etc /usr, unowned files	~8s
17f	cert-audit	openssl certificate expiry, SSH host key types	~5s
17g	net-security	arp duplicate IPs, ICMP redirect, TCP wrappers	~2s
17h	auditd	auditctl -l rule count, auditd.conf rotation config	~2s
17i	open-files	lsof deleted executables, world-writable sockets	~3s
17j	mem-security	/proc/swaps encryption status, vm.overcommit	~2s
17k	pam-security	pam module integrity, TOTP presence, pam_wheel	~2s
17l	fs-hardening	mount noexec/nosuid /tmp /dev/shm, home dir perms	~2s
17m	container	docker inspect User field, AppArmor/seccomp profiles	~3s
17n	repo-sec	trusted=yes overrides in apt sources, 3rd-party repos	~2s
17o	time-sync	ntp/chrony/timesyncd active, drift, server config	~2s
17p	ipv6	ip6tables count, UFW IPv6, RA accept sysctl	~2s
17q	ssh-extras	Ciphers/MACs/KexAlg from cached sshd -T output	<1s
17r	core-dump	kernel.core_pattern, fs.suid_dumpable, systemd core	~2s
17s	systemd	PrivateTmp/ProtectSystem/NoNewPrivileges per service	~5s
17t	sudo-audit	sudoers NOPASSWD, wildcard commands, file perms	~2s
17u	log-integrity	remote syslog forward config, logrotate retention	~2s
17v	compilers	gcc/clang/build-essential, gdb/strace/ltrace	~1s
17b3	hw-security	Secure Boot (mokutil), IOMMU, CPU vuln mitigations, TPM	~4s
17b4	boot-sec	GRUB password_pbkdf2, cmdline flags, /boot permissions	~2s
17b5	web-server	nginx server_tokens/TLS/headers, Apache ServerTokens	~3s
17b6	secrets	*.pem world-readable, .env credentials, bash history	~6s
17w	net-ifaces	PROMISC flag, ARP anomalies, packet drop counters	~2s
17x	kernel-mods	dangerous lsmod, module signing enforcement, blacklist	~2s
17y	mac-profiles	AppArmor enforce/complain count, recent denials	~2s
17z	exposure	consolidate all open ports with CRIT/HIGH/MED/LOW risk	~1s

7. The HMAC-Signed Archive

After each scan, all output files are bundled into a ZIP archive with an HMAC-signed manifest. The verify command detects any modification to any file.

7.1 INTEGRITY.txt Manifest Format

```
# Wowscanner integrity manifest v2 # Generated : 2026-05-10 14:32:15 # Host : myserver # Format : SHA256 SHA512 SIZE MTIME MODE UID GID filename # a3f8d2c1... 9e4b7f3a... 45231 1746882735 0o100644 0 0 report.txt ... HMAC-SHA256: f9e8d7c6b5a43210... ← machine-derived HMAC sealing the manifest
```

7.2 Machine-Derived HMAC Key

```
def machine_key(): parts = [socket.gethostname()] for p in ["/etc/machine-id", "/var/lib/dbus/machine-id"]: try: parts.append(open(p).read().strip()); break except: pass return hashlib.sha256("|".join(parts).encode()).digest() # Why machine-key (not passphrase): # verify must run without passphrase → machine-key is deterministic # Archives are machine-specific — an archive from server A will not # verify on server B (different machine-id = different HMAC key)
```

8. All Commands Reference

(none) — Run full security audit

```
sudo wowscanner [FLAGS]
```

Starts all 54 audit sections. Passphrase required. Reports written to current directory.

clean — Remove output files

```
sudo wowscanner clean [--all|--integrity|--full]
```

--all also removes /var/lib/wowscanner/ data. --full = factory reset.

verify — Verify archive integrity

```
sudo wowscanner verify [--reset-history]
```

Checks SHA-256+SHA-512 of every archived file vs INTEGRITY.txt. No passphrase needed.

diff — Compare two most-recent scans

```
sudo wowscanner diff
```

Shows new FAILs, resolved FAILs, new WARNs, resolved WARNs, and score delta.

harden — Apply kernel hardening

```
sudo wowscanner harden
```

Writes /etc/sysctl.d/99-wowscanner.conf and applies ASLR, dmesg, kptr, syncookies, etc.

baseline — Snapshot PASS findings

```
sudo wowscanner baseline
```

Saves all current PASSes to findings_last.db. Next scan flags regressions.

install-timer — Install weekly systemd timer

```
sudo wowscanner install-timer
```

Creates wowscanner.service + wowscanner.timer. OnCalendar=weekly, RandomizedDelaySec=3600.

remove-timer — Remove systemd timer

```
sudo wowscanner remove-timer
```

recover — Wipe auth key (lost passphrase)

```
sudo wowscanner recover
```

Backs up auth.key, then deletes it + script.sig. Next run triggers setup wizard.

reset-auth — Change passphrase / reset

```
sudo wowscanner reset-auth [forgot|rk|--force]
```

forgot/rk: use 48-char recovery key. --force: full wipe of all auth data + history.

8.2 Flags

Flag	Effect
--no-pentest	Skip sections 0a–0e (active pentest). Recommended for routine scans.
--fast-only	Skip pentest + minimal timeouts. ~2–4 minutes.
--no-lynis	Skip section 15 (Lynis CIS audit).
--no-rkhunter	Skip section 14b (rkhunter + chkrootkit).
--quiet	Suppress [■ INFO] lines in terminal output.
--email=ADDRESS	Send archive by email after scan (requires msmtp/sendmail).
--webhook=URL	POST JSON summary {hostname, score_pct, pass, fail, warn, rating} after scan.

9. Password Recovery — All Paths

Path	When	Command	Effect
Path 1	You know passphrase, want to change it	sudo wowscanner reset-auth	Prompts current passphrase, then new (twice). Data key preserved →
Path 2	Forgot passphrase, have 48-char recovery key	sudo wowscanner reset-auth forgot	Prompts recovery key, then new passphrase. Data key preserved.
Path 3	Forgot passphrase AND recovery key	sudo wowscanner recover	Backs up auth.key → auth.key.bak, deletes auth.key + script.sig. New
Path 4	Complete factory reset	sudo wowscanner reset-auth --force	Deletes auth.key, script.sig, backup, AND /var/lib/wowscanner/. No ur

10. Remediation Guide for Common Failures

10.1 SSH Hardening

```
# Edit /etc/ssh/sshd_config: PermitRootLogin no PasswordAuthentication no MaxAuthTries 3
AllowAgentForwarding no X11Forwarding no ClientAliveInterval 300 AllowUsers operator admin Ciphers
chacha20-poly1305@openssh.com,aes256-gcm@openssh.com MACs hmac-sha2-512-etm@openssh.com KexAlgorithms
curve25519-sha256 # Apply: sudo systemctl reload sshd
```

10.2 Kernel Hardening

```
sudo wowscanner harden # applies all sysctl settings at once
```

10.3 Firewall

```
sudo ufw default deny incoming sudo ufw allow ssh sudo ufw --force enable
```

10.4 Fail2Ban

```
sudo apt-get install fail2ban sudo systemctl enable --now fail2ban
```

10.5 Secrets & Credential Exposure

```
# Fix world-readable private keys: find /etc /home /root /var/www -name "*.pem" -o -name "id_rsa" | xargs
chmod go-r # Clear sensitive bash history: history -c && cat /dev/null > ~/.bash_history
```

10.6 Kernel Module Blacklisting

```
cat > /etc/modprobe.d/hardening.conf << EOF install dccp /bin/true install sctp /bin/true install rds
/bin/true install tipc /bin/true install firewire-core /bin/true install cramfs /bin/true EOF sudo
update-initramfs -u
```

11. Legal Notice & License

Penetration Testing Warning

- Sections 0a–0e use REAL attack tools that perform active exploitation
- ONLY run on systems you OWN or have EXPLICIT WRITTEN PERMISSION to test
- Unauthorised use may violate computer fraud laws in your jurisdiction
- Skip all pentest sections with: sudo wowscanner --no-pentest
- The author accepts NO LIABILITY for misuse

BSD 2-Clause License

Author	cocoflan
Repository	https://github.com/cocoflan/wowscanner
License	BSD 2-Clause
Version	2.4.0
Script size	17,584 lines of Bash